

Understanding constexpr in C++

Compile-Time vs. Runtime Evaluation

Introduction

When writing C++ code using `constexpr`, a common point of confusion is whether to assign the result of a `constexpr` function to a standard variable or a `constexpr` variable. Consider the two snippets below:

Snippet 1: Using a Standard `int`

```
#include <iostream>
using namespace std;

constexpr int product(int x, int y) {
    return x * y;
}

int main() {
    int result = product(5, 10);
    std::cout << result << std::endl;
    return 0;
}
```

Snippet 2: Using a `constexpr int`

```
#include <iostream>
using namespace std;

constexpr int product(int x, int y) {
    return x * y;
}

int main() {
    constexpr int result = product(5, 10);
    std::cout << result << std::endl;
    return 0;
}
```

Both snippets are completely correct and valid C++ code, but they behave differently under the hood depending on when the mathematics actually occurs. To understand this, we must define the two primary phases of a program's lifecycle.

The Two Phases: Compile Time vs. Runtime

- **Compile Time:** This is when the source code is translated into machine code by the C++ compiler. The program is not actively running yet.

- **Runtime:** This is when the compiled executable is actually launched and executing. The program is now live, interacting with the operating system, taking inputs, and computing dynamic values.

Snippet 2: The “Zero Overhead” Guarantee

```
constexpr int result = product(5, 10);
```

When you apply `constexpr` to the **variable**, you are issuing a strict command to the compiler: *“You must evaluate this expression right now, during Compile Time, before the program ever runs.”*

Because you provided hardcoded numbers (5 and 10), the compiler can calculate the result immediately. Instead of generating machine code that instructs the CPU to multiply these two numbers, the compiler simply writes the resulting value (50) directly into the final executable.

What this means: During Runtime, the computer does not have to perform any arithmetic. It instantly outputs 50. This is the essence of **zero runtime overhead**—shifting the computational burden from the executing program to the compilation process.

The Catch: Because you forced the compiler to evaluate the result early, you *cannot* use variables that change at runtime. If you attempt to use user input, the compiler will throw an error because it cannot predict what the user will type:

```
int x;
cin >> x;
// ERROR: The compiler cannot predict 'x' at compile time!
constexpr int result = product(x, 10);
```

Snippet 1: The Flexible Approach

```
int result = product(5, 10);
```

By using a standard `int`, you provide the compiler with flexibility. You are saying: *“Store this result. If you can optimize it and calculate it during Compile Time, great. If you must wait until Runtime, that is also fine.”*

Because `product()` is declared as a `constexpr` function, it has the unique ability to execute in **both** phases. If you provide hardcoded numbers, modern compilers with optimizations enabled will likely resolve it to 50 internally.

However, unlike Snippet 2, Snippet 1 gracefully handles dynamic, runtime data:

```
int x, y;
cin >> x >> y; // User types inputs at Runtime
int result = product(x, y); // Perfectly valid
```

In this scenario, because the compiler cannot predict `x` and `y`, the multiplication occurs live while the program is running.

Summary

- **Use Snippet 2 (constexpr variable):** When you know the inputs at compile time, want to enforce a hardcoded calculation, and want to absolutely guarantee zero runtime overhead.

- **Use Snippet 1 (Standard variable):** When the inputs to the function might eventually be determined at runtime (e.g., from a user, file, or database), as a `constexpr` variable cannot store a runtime-evaluated value.